WEEK 6 · BUILD PHASE BEGINS

Lock scope. Set up. Then build.

The gate is passed — you proved the **right thing**. BUILD asks a new question: can you finish it? Today you **lock scope**, run **Sprint 0**, and give your AI agents real hands with **MCP**.

BUILD · month 2 of 4 | Scope lock · Sprint 0 · repo · MCP | In-class case due today

Dr. Prasara Jakkaew · ADT MFU

RECAP · THE PHASE CHANGES TODAY

DISCOVER is done — the danger just changed

W1 · Charter → W2 · Ethics + law → W3 · Backlog → W4 · Design → **W5 · GATE ✓**

→ **W6 · BUILD**

DISCOVER ASKED

"Are we building the right thing?"

The risk was **building the wrong software**. The gate protected you from it.

BUILD ASKS

"Are we building the thing right?"

The new risk is **never finishing the right thing** — drowning in features, no setup, one month gone.

💡 **IN PLAIN TERMS** You have **one month** to ship **one workflow** a real user can finish. Everything today exists to protect that month.
🇹🇭 มีเวลา 1 เดือน ทำ 1 เวิร์กโฟลว์ให้ผู้ใช้จริงทำจนจบได้ ทุกอย่างวันนี้คือการปกป้องเดือนนั้น

THE NUMBER-ONE WAY ONE-MONTH BUILDS DIE

Meet the enemy: scope creep

A REAL WEEK-6 STORY · LINGUALOOP

- Passed the gate with a **language-flashcard app**
- Sponsor: *"add video calls… add a marketplace… add a social feed"*
- Team feels pressure to **say yes to everything**
- One month later: **five half-built features, nothing ships**

WHAT THIS ACTUALLY IS

Scope creep — not "iteration."

New requests aren't bad. Saying **yes to all of them mid-build** is what kills the timeline. The cure isn't refusing people — it's a **locked scope** you can point to.

THE TRAP "The sponsor is paying, so we must add it." No. Your job is to **ship one thing that works**, then decide what's next — **with** the sponsor, on the record.
🇹🇭 **Scope creep** = งานบวมขึ้นเรื่อย ๆ จนไม่เสร็จสักอย่าง วิธีแก้ไม่ใช่ปฏิเสธคน แต่คือมี **ขอบเขตที่ล็อกไว้** ให้ชี้ได้

THE TOOL · MOSCOW

Locking scope = drawing four lines

MUST

Ship or fail

Your **one core workflow**. If this isn't done, there is no product. Keep it **small**.

SHOULD

Important, not vital

Adds real value — but the product still works without it. Only if Musts are done.

COULD

Nice to have

Do it only if there's spare time. First to be dropped.

WON'T

Not this build

Written down, on purpose. The video calls, the marketplace — parked, not forgotten.

EXAMPLE · A FOOD-DELIVERY APP **MUST** place an order · **SHOULD** track the driver live · **COULD** rate the restaurant · **WON'T** in-app chat with the driver. Same app, four honest verdicts — no feature is "bad," each just has a place.

THE POINT OF THE WON'T LIST Scope lock isn't just picking what to build — it's **explicitly naming what you will not build this month**, so no one has to argue about it again mid-sprint.
🇹🇭 หัวใจอยู่ที่ช่อง **WON'T** — เขียนสิ่งที่ "จะไม่ทำเดือนนี้" ให้ชัด จะได้ไม่ต้องมาเถียงกันซ้ำกลางสปรินต์

HOW TO ACTUALLY LOCK IT

The one-workflow rule

DO THIS

- Pick the **single workflow** a real user completes **end to end**
- Trace it: it must answer a **validated pain** from the gate
- Everything else → Should / Could / **Won't**
- Write the locked list where the whole team sees it

CUPLOG · LOCKED

- **MUST** — log a cup in ≤3 taps · see today's total
- **SHOULD** — edit a mistaken entry
- **COULD** — export the day as CSV
- **WON'T** — multi-café, payroll, inventory, accounts

TEST IT · ONE BREATH Read your MUST list aloud in one breath.
❌ **Too big:** "log a cup, edit it, export CSV, see weekly charts, and sync across cafés."
✅ **Locked:** "log a cup in ≤3 taps and see today's total." Everything cut just moved to Should / Could / Won't — nothing was thrown away.

THE SPONSOR KEEPS ASKING · WHAT TO SAY

Not "yes." Not "no." — "Not now, and it's recorded."

WEAK RESPONSES

- "Sure!" → scope creep, timeline dies
- "No." → damages the relationship
- Silently ignore it → sponsor loses trust
- Pause the build until requests stop → they never stop

THE ENGINEER'S MOVE

- "Great idea — it goes on the **Could / Won't list** for this build."
- Record it in the backlog as **out-of-scope (this sprint)**
- Keep the **one MUST workflow** locked
- Revisit the parked list **after** Alpha (W8)

💡 **IN PLAIN TERMS** You don't reject the sponsor — you reject **doing it right now**. A written "not this build" is a promise you'll come back to it, not a rejection.

BEFORE THE FIRST FEATURE · SPRINT 0

Sprint 0 = build the foundations, not the features

SPRINT 0 IS

- Setup **before** feature sprints start
- Repo, project board, branches, environment
- The agent's rulebook: `CLAUDE.md` + `rule.md`
- A "definition of done" everyone agrees on

SPRINT 0 IS NOT

- Shipping the first paid version
- Final user-acceptance testing
- Writing the launch / marketing plan
- Building features already (that's W7+)

WHY IT MATTERS Teams that skip Sprint 0 and "just start coding" spend week 7 fixing their own setup. One clean day now buys you three clean weeks.
🇹🇭 ทีมที่ข้าม Sprint 0 แล้วรีบโค้ด มักเสียเวลาสัปดาห์ 7 ไปแก้ระบบตัวเอง · ลงทุน 1 วันวันนี้ ได้คืนมา 3 สัปดาห์

SPRINT 0 · THE REPO-SETUP CHECKLIST

What "set up" concretely means

YOUR REPO AT END OF TODAY

- GitHub repo created · whole team has access
- `CLAUDE.md` + `rule.md` at **repo root** (from W2/W3)
- Project board with the **locked MUST items** as issues
- `main` + feature branches · a simple branch rule
- `README.md` skeleton · `.gitignore`

repo root

```
CLAUDE.md        # how the agent works here
rule.md          # legal/PDPA rules (W2)
README.md        # what · how to run
.gitignore
docs/            # spec, backlog, diagrams (W3–4)
src/             # the one workflow — nothing else yet
```

THE BOARD = THE LOCK, MADE VISIBLE Only **MUST** items become "To do" issues now. Should/Could/Won't live in a separate parked column — visible, but not in this sprint.

AI-FIRST · THE BIG IDEA FOR BUILD

AI is for the whole SDLC — not just the coding

SDLC PHASE	WHERE YOUR AI AGENTS HELP	YOU ALREADY DID THIS IN…
Discover	Synthesise interviews, cluster pains, draft requirements	W3 requirement-writer
Design	Draft diagrams, design system from a spec + screen	W4 design-with-agents
Build	Vibe-code the workflow, generate boilerplate	W6–8 (now)
Test	Generate test cases, edge cases, usability scripts	W7+, TEST phase
Docs	README, API notes, the case-study write-up	Portfolio pack

ONE CONCRETE PROMPT (TEST PHASE) "From this SRS, generate **8 edge-case tests** for the log-a-cup flow — empty name, 100 cups in one minute, offline save." No code autocompleted — that's AI doing **test design**.

THE EXAM-WORTHY LINE Using AI **only** to autocomplete code lines is the **narrow** use. The AI-First team uses it for **discovery synthesis, test generation, and docs too**.

THE ENABLER · MODEL CONTEXT PROTOCOL

MCP gives your agent hands and eyes

WHAT IT IS

- A **standard way** for an AI agent to connect to **external tools & data**
- Turns "a chatbot that talks" into "an agent that **acts**" — reads your repo, opens a browser, queries a database
- One protocol, many **servers** (GitHub, browser, docs, files…)

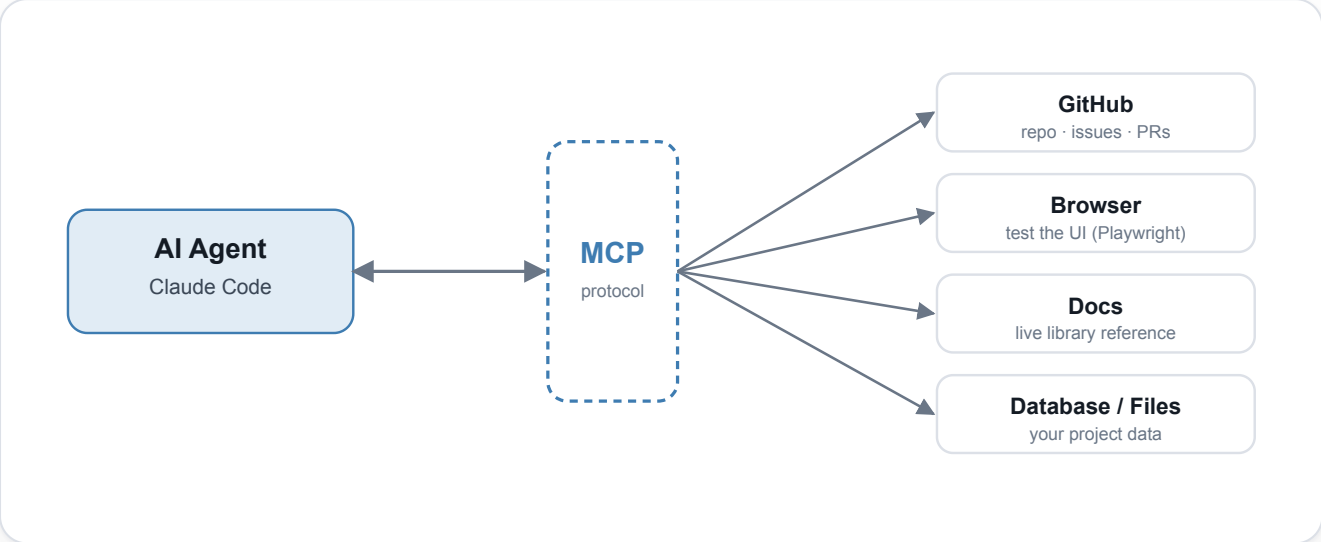
WHAT IT IS NOT

- Not a faster compiler
- Not a replacement for Git / version control
- Not "automatic PDPA encryption"

💡 **IN PLAIN TERMS** Without MCP, the agent can only **describe** what to do. With MCP, it can **reach into your real tools and do it** — across the whole SDLC, not only coding.
🇹🇭 ไม่มี MCP = เอเจนต์ได้แค่ "บอกวิธี" · มี MCP = เอเจนต์ "ลงมือทำ" กับเครื่องมือจริงของเราได้เลย

MCP · THE SHAPE OF IT

One agent → one protocol → many tools



TODAY YOU CONNECT ONE Start with **GitHub MCP** — it plugs the agent straight into the repo and board you set up in Sprint 0. Add browser / docs later as the build needs them.

MCP · ONE REAL MOMENT

Same question — with and without MCP

WITHOUT MCP

You: *"List the open issues on our repo and tell me which are done."*

Agent: "I can't see your GitHub. Paste the issue list here and I'll help." → you copy-paste by hand, every time.

WITH GITHUB MCP

Same prompt. The agent reads your board itself and answers:

```
# issues on CupLog · read live via MCP
#12  Log a cup in ≤3 taps        open
#13  Show today's total          open
#14  Set up .gitignore           done → ready to close ✔
```

THE DIFFERENCE Nothing about the prompt changed — **the agent's reach did**. That is the whole point of MCP: same conversation, real hands.

🇹🇭 คำถามเดิมเป๊ะ ต่างกันแค่เอเจนต์ "เอื้อมถึงเครื่องมือจริง" ได้หรือไม่

HANDS-ON · WHAT EVERY COMPANY DOES TODAY

Three things, in this order

1 LOCK

MoSCoW your backlog

As a team, sort every backlog item into Must / Should / Could / **Won't**. Read the MUST list aloud — one breath, one workflow.

2 SET UP

Run Sprint 0 on the repo

Create the GitHub repo, drop in `CLAUDE.md` + `rule.md`, turn MUST items into board issues, set the branch rule.

3 CONNECT

Wire up one MCP

Connect **GitHub MCP** to Claude Code and prove it works — ask the agent to list your open issues. If it reads them, you're connected.

PROOF OF DONE By the end of class: a locked MoSCoW board, a live repo with the two rule files, and an agent that can see your issues. That's your BUILD launchpad.

MOVING FAST ≠ NO RULES

BUILD does not cancel Week 2

TEMPTING, AND WRONG

- "Coding started — delete `rule.md`."
- Hide which parts the AI wrote
- Skip PDPA "until it works"
- Stop caring about consent / logs

WHAT YOU ACTUALLY DO

- **Keep `rule.md`** — the agent follows it while coding
- **Disclose** your AI use — it's a strength, not a secret
- PDPA / CCA §26 / ETA stay **binding** in BUILD
- Stay inside your governance rules as you speed up

LINK TO W2 The faster the agent moves, the **more** your `rule.md` earns its place — it's what keeps a fast build legal and honest.

ONE DECISION THE LOCK MAKES FOR YOU

Locked scope → the simplest architecture that works

JUSTIFIED BY A ONE-WORKFLOW, ONE-MONTH BUILD

- A **single, well-structured app** (a "modular monolith")
- A **managed** backend / database — don't run your own
- One clear **seam** around the core workflow (recall W4)

NOT JUSTIFIED — THIS MONTH

- Microservices for a 5-person, 1-month project
- Custom infra you have to babysit
- Architecture "for scale you don't have yet"

THE PRINCIPLE **Scope drives architecture.** You locked scope to one workflow — so the right architecture is the smallest one that ships it well, not the most impressive one.

HOW TODAY RUNS · 180 MIN

Teach → lock → set up → connect → case

1 0–35 MIN

Scope, Sprint 0 & MCP — taught

These slides. Scope creep, MoSCoW, Sprint 0, AI across the SDLC, what MCP is. **You're here.**

2 35–75 MIN

Lock scope (MoSCoW)

Each company sorts its backlog into M/S/C/W and writes the WON'T list. I walk the room and pressure-test every MUST list.

3 75–125 MIN

Sprint 0 · repo + board

Repo live, `CLAUDE.md`+`rule.md` in, MUST items as issues, branch rule set. (10-min break inside this block.)

4 125–160 MIN

Connect one MCP

Wire GitHub MCP to Claude Code; prove it lists your issues. First AI-assisted setup task on the real repo.

5 160–180 MIN

In-class case + close

Submit today's in-class case. Preview W7 architecture studio + build sprint.

IN-CLASS CASE · DUE TODAY · GRADED

The scope-creep decision

SCENARIO

SnapCart passed the gate with one workflow: *"scan a barcode → see the cheapest nearby price."* In week 6, their sponsor asks for a loyalty-points system, an in-app chat, and a delivery feature — "by next week." The team has one month and five people.

Deliver 1

MoSCoW board

Sort the original workflow + the 3 new requests into M/S/C/**W**.

Deliver 2

The WON'T line

One sentence you'd say to the sponsor — polite, on the record.

Deliver 3

Sprint 0 steps

List the 4–5 setup steps SnapCart does **before** writing any feature.

SUBMIT One page per company to Google Classroom before you leave. This is your **in-class case** (part of the 5% in-class mark) — it's about judgement, not length.

THE FIVE WEEK-6 TRAPS

Know them, dodge them

THE TRAP	WHAT IT LOOKS LIKE	THE FIX
Say-yes-to-sponsor	Every request becomes a MUST	Park it on Could / Won't; keep one workflow
Too many MUSTs	The MUST list is a paragraph, not a breath	Cut until one workflow remains
Skip Sprint 0	"Let's just start coding"	Repo, board, rule files first — one clean day
AI = autocomplete only	Agent used for code lines, nothing else	Use it for tests, docs, synthesis too
Drop governance	Delete `rule.md`, hide AI use	Keep rule.md; disclose AI; PDPA still binds

THE MODEL · A LOCKED, SET-UP BUILD

What a good W6 looks like — CupLog

Scope · locked

MUST log a cup + daily total. **WON'T** multi-café, payroll, inventory — written in the backlog, not argued about again.

Sprint 0 · done

Repo live, `CLAUDE.md`+`rule.md` at root, 6 MUST issues on the board, `main`+feature branches.

MCP · connected

GitHub MCP wired to Claude Code — the agent lists and closes issues on CupLog's own board.

Governance · kept

`rule.md` still guides the agent: PDPA consent for staff names, CCA §26 90-day logs — enforced while coding.

WHERE TO STUDY IT Full worked example in **CupLog-Worked-Example-2569** (§1 repo layout, §2 `CLAUDE.md`+`rule.md`). Copy its *structure*, not its content.

ACTIONS · BY THE END OF TODAY

Leave class ready to build

EVERY COMPANY

- **MoSCoW locked** — MUST + an explicit WON'T list
- **Repo live** · board with MUST issues · branch rule
- `CLAUDE.md` + `rule.md` at repo root
- **One MCP connected** (GitHub) and proven
- In-class case submitted to Google Classroom

BETWEEN NOW AND W7

- Start the **first feature sprint** on the MUST workflow
- Keep the WON'T list closed — no new MUSTs
- Bring your repo + board to **W7 (Sep 23)** architecture studio

REMINDER BUILD is graded inside the Project. **Alpha Demo is W8 (Oct 7)** — a real user finishing your one workflow. Everything you lock today points at that day.

WEEK 6 · CLOSE

Lock it, set it up, then build.

Passing the gate earned you the right to build. Finishing takes discipline: **one locked workflow**, a **clean Sprint 0**, agents with real hands through **MCP**, and governance you never dropped. Now go make the thing — **right**.

Next · W7 (Sep 23): Architecture studio · build sprint

Alpha Demo · W8 · Oct 7

Scope stays locked.